SESSION 1

Foundations: LLMs & the Anatomy of a Prompt

1

How a language model produces text — and how a prompt steers it.

Mardi 15 septembre · 8h00–9h50 · 1h50

Prompt & Context Engineering — Stefan Duprey

TODAY

Session 1 — Git + PE kickoff

You will be able to

- Use Git: init, add, commit, and read history
- Explain how an LLM generates text, token by token
- Identify the four parts of a good prompt
- Set up your repo and run a local model

RUN OF SHOW

0:00	Intro + course map
0:15	Git foundations + hands-on
1:00	LLM mental model & prompt anatomy
1:35	Lab 0 — setup, commit, first prompts

COURSE MAP

Where this course is going

Here's the shape of the whole journey, so you always know where a given day fits.

F Foundations

Sessions 1–2. How models work, and the everyday prompting toolkit.

M Measure

Sessions 3–4. Evaluation first, then reasoning and structured output.

C Context

Sessions 5–6. Retrieval, tools and memory as engineered context.

P Production

Sessions 7–9. Cost, safety, and a capstone you present.

Why version control

- Snapshots of your work you can return to anytime
- A safety net — experiment without fear of breaking things
- Branches let you try ideas in isolation
- The backbone of every team collaboration

WHAT IS GIT

A distributed version-control system: every clone is a full copy of the whole history.

• GIT

The everyday Git loop

1

Edit

Change files in your working directory.

2

Stage

git add — choose exactly what goes into the next snapshot.

3

Commit

git commit -m "..." — save the snapshot with a message.

4

Repeat

Small, meaningful commits beat one giant one.

• GIT

See what changed — and hide what shouldn't

- git status — what's staged, modified, untracked
- git log --oneline — the history at a glance
- git diff — the exact lines that changed
- .gitignore — keep secrets and junk out of the repo

terminal

```
git status
git add report.md
git commit -m "draft summary"
git log --oneline

# .gitignore
.env
__pycache__/
.ipynb_checkpoints/
```

MENTAL MODEL

An LLM is a next-token predictor

Before any technique makes sense, let's be honest about the surprisingly simple thing the model is doing when it 'writes'.

- **It predicts the next token, over and over**

Given the text so far, the model outputs a probability for every possible next token, picks one, appends it, and repeats. That single loop produces everything it writes.

- **There is no lookup and no memory**

It isn't searching a database or recalling your last chat. Each call starts fresh from whatever text you put in front of it.

- **So the prompt is the whole program**

Steering the model means steering that text. Everything it can use for your task has to be present in the context you send.

KEY IDEA

The prompt is the model's entire working memory. If it isn't in the context, the model cannot use it — no matter how obvious it seems to you.

TOKENS

Text is split into tokens, not words

The model doesn't read words the way you do — it reads tokens, and that one fact explains a lot of odd behaviour.

- **Models read tokens**

A token is a chunk of text — often a word piece. The model never sees raw characters or whole words the way you do.

- **Roughly 4 characters per token**

A useful rule of thumb in English: ~¾ of a word. Numbers, code and rare words split into more tokens.

- **You pay and budget in tokens**

Cost, speed and the context-window limit are all counted in tokens — so token count is a real engineering quantity, revisited in S7.

tokenization

```
# the string ...
"Prompt engineering"

# ... becomes tokens
["Prom", "pt", " engineering"]

budget = context_window - prompt_tokens
```

DECODING KNOBS

What you can actually turn

Beyond the prompt text you get a handful of dials — here's what each one really does.

T Temperature

0 makes the model pick the most likely token every time — focused and repeatable. Higher values add randomness and variety.

P Top-p

Samples only from the smallest set of tokens whose probabilities sum to p. A different way to control how adventurous the output is.

N Max tokens

Caps how long the response can be. Protects latency and cost — and stops runaway generations.

|| Stop sequences

Strings that force generation to halt. Useful for keeping output inside a format you can parse.

LIMITS

The context window is a hard budget

Think of the context window as a fixed desk: everything you want the model to use has to fit on it at once.

- **Prompt and answer share one budget**

The window is a fixed number of tokens for everything: your instructions, examples, retrieved data, and the model's reply all compete for the same space.

- **Everything you add costs room**

Long instructions and big pasted documents leave less space for the answer — and slow the call down.

- **Position matters, not just size**

Models attend least to the middle of a long input. We'll exploit and defend against this in S5 (context rot).

RULE OF THUMB

Put the key instruction first or last — not buried in the middle of a wall of text the model will skim.

FAILURE MODE

Why models hallucinate

Once you see the model as a plausibility engine, confident wrong answers stop being mysterious and start being manageable.

- **They optimise for plausible, not true**

The training objective rewards text that looks like a likely continuation. A confident wrong answer is often more 'likely-looking' than 'I don't know'.

- **Gaps get filled with guesses**

With no grounding, the model invents specifics — names, numbers, citations — that fit the pattern but aren't real.

- **The fixes are structural**

Ground the model in real sources (S5), let it say 'I don't know', and measure with an eval set (S3). You reduce hallucination; you don't switch it off.

PREVIEW

Grounding (S5) and evaluation (S3) are how we fight this systematically — not by asking the model nicely to stop.

STRUCTURE

Every prompt is made of roles

A prompt isn't one blob of text; it's a little conversation with distinct parts, and naming them gives you control.

1

System

The standing instructions: who the model is, the rules it must follow, and the output format. Set once, applies to the whole exchange.

2

User

The actual request, plus any input or data the model should work on for this turn.

3

Assistant

The model's reply — and, when you send history, its previous replies become part of the context for the next turn.

LIVE DEMO

The same task, two prompts

Let's watch specificity do its work — the same request, two very different results.

✗ Vague

- "Summarize this."
- No audience — who is it for?
- No length or format target
- No rule for missing information
- Result changes every run

✓ Specific

- Role: "You are an editor."
- Task + exactly 3 bullets, max 60 words
- Audience: a non-technical manager
- "If anything is unclear, ask one question."
- Result is consistent and usable

HANDS-ON LAB

Lab 0 — Setup & first prompts

LOCAL · OLLAMA

prompt-engineering-course/01_foundations

TASKS

1. Fork & clone the repo; create and activate a virtual environment
2. Install Ollama and pull the model: ollama pull llama3
3. Make your first commit once setup runs (this is the Git touch-in)
4. Open 01_tokenization.ipynb; run the same prompt at temperature 0 and 1

DELIVERABLE

A working local setup with a first commit on your fork, and one prompt run at two temperatures — note what changed.

WRAP-UP

Recap — Session 1

- ✓ An LLM predicts one token at a time, with no memory
- ✓ The prompt is the model's entire working memory
- ✓ Decoding knobs trade focus for creativity
- ✓ Specific prompts (role, task, format) beat vague ones

NEXT SESSION

Techniques & task patterns

The everyday toolkit — zero/few-shot, roles, structure — and how prompting changes by task: classification, extraction, summarization.